

The open AI alternative

How ML Engineers Build It

Sergio Paniego Blanco (@sergiopaniego)

Machine Learning Engineer @ Hugging Face



Content

- What's HF?
- Open vs Closed AI
- Does open AI makes sense?
- The role of an ML Engineer at HF
- Building AI with HF tools



Hi, I'm Sergio 🙌

- Machine Learning Engineer @ Hugging Face
 - Multimodality, Post-Training & Agents
- PhD in AI
- Open source and AI
 - GSoC (JdeRobot)
 - ML Circle
- Teaching and Sharing



Sergio Paniego Blanco (@sergiopaniego)



What's Hugging Face? 🤗

AI open platform for building, sharing and deploying AI (HF Hub):

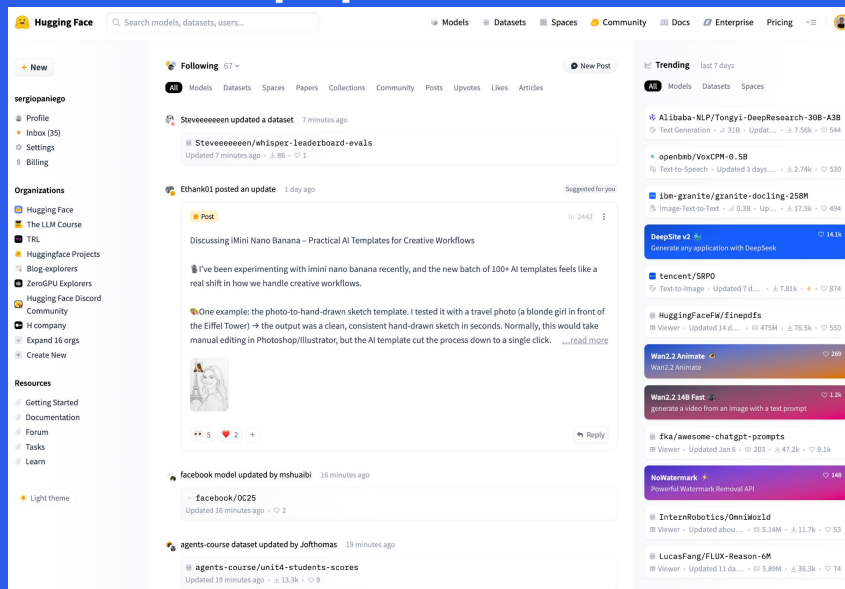
- Models, datasets, apps...
- Posts, blogs, courses, papers, leaderboards...

2.1M

public models

500K+

public datasets



1M+

daily downloads

1M+

daily visitors

What's Hugging Face? 🤗

Set of open source projects to empower AI development and advancement



Transformers



Datasets



Trackio



Hub



30+

Libraries



Transformers.js



Diffusers

timm



Accelerate



PEFT



smolagents

Open vs Closed



	Open	Closed / Proprietary
Security	Models can be self-hosted , data stays in your environment	Models cannot be self-hosted. Data is sent outside your environment to vendor
Control	The timing and nature of updates are controlled by you	Updates and changes to performance can happen without notice
Customization	Full source code access to customize the model for your needs	Limited ability to customize for your needs
Transparency	Inspect code and data provides better auditability and understandability	No ability to audit or understand performance
Cost	Typical lower long term cost due to smaller model size	larger model size and proprietary premium often balanced by decreased cost from server-side optimization
Latency	Lower latency due to on premise and smaller model sizes	Often greater latency due to larger model sizes + API latency
Quality	No single approach is best. Each use case will vary. Proprietary is typically closer to the frontier of performance .	
Examples	 OpenAI  Meta  deepseek  MISTRAL AI  Microsoft	 OpenAI ANTHROPIC

How HF democratizes ML/AI



ML engineers who
train models from
scratch

Software engineers
who use inference
APIs, ML engineers
who fine-tune
models

Anyone who
can use a GUI



Datasets



Models



Spaces

Used everywhere in the AI world

15,000+ startups and enterprises



Open source contributors



**Hugging
Face**

Cloud partners



Hardware partners



On-prem partners



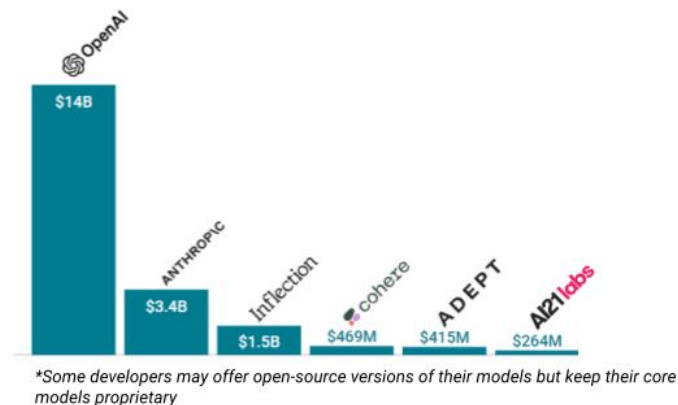
Is the market interested?



The private market is split into open vs. closed

Disclosed equity funding to LLM developers (as of 10/27/2023)

Closed-source LLMs



Open-source LLMs



*Excludes open-source developers that have not raised equity funding

Is this actually possible?

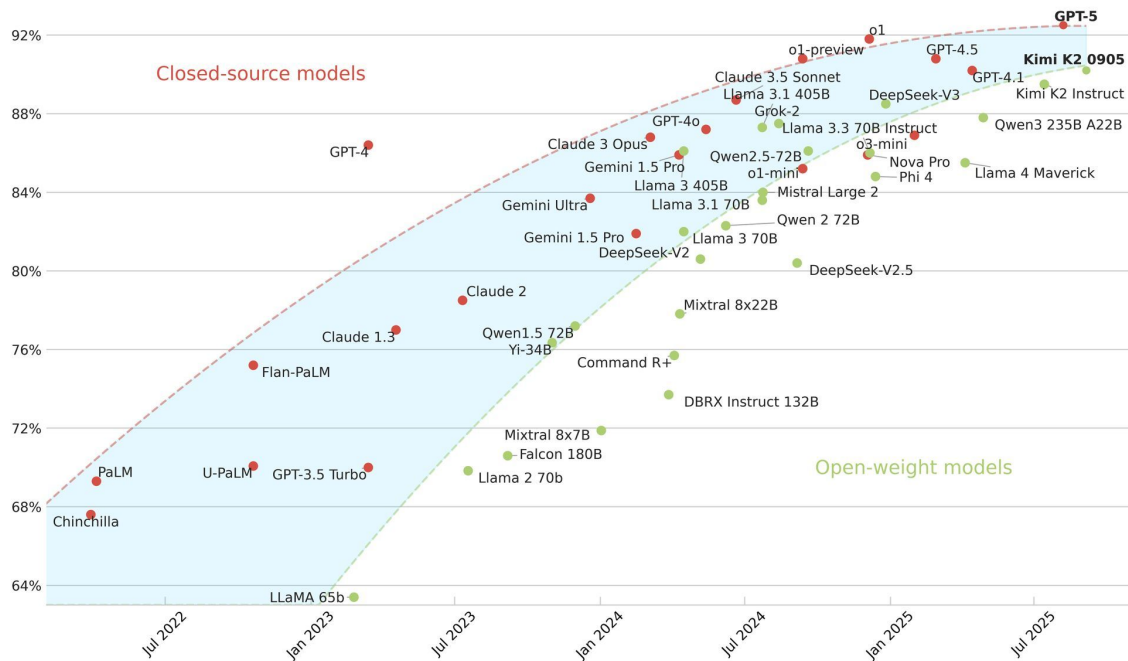


Closed-source vs. open-weight models

@maximelabonne

Chinese MoE models now dominate frontier open-source releases.

MMLU (5-shot)



The role of a ML Engineer at HF

It depends on your specific team:

- Transformers (or any particular project)
- Research
- Advocacy
- ...

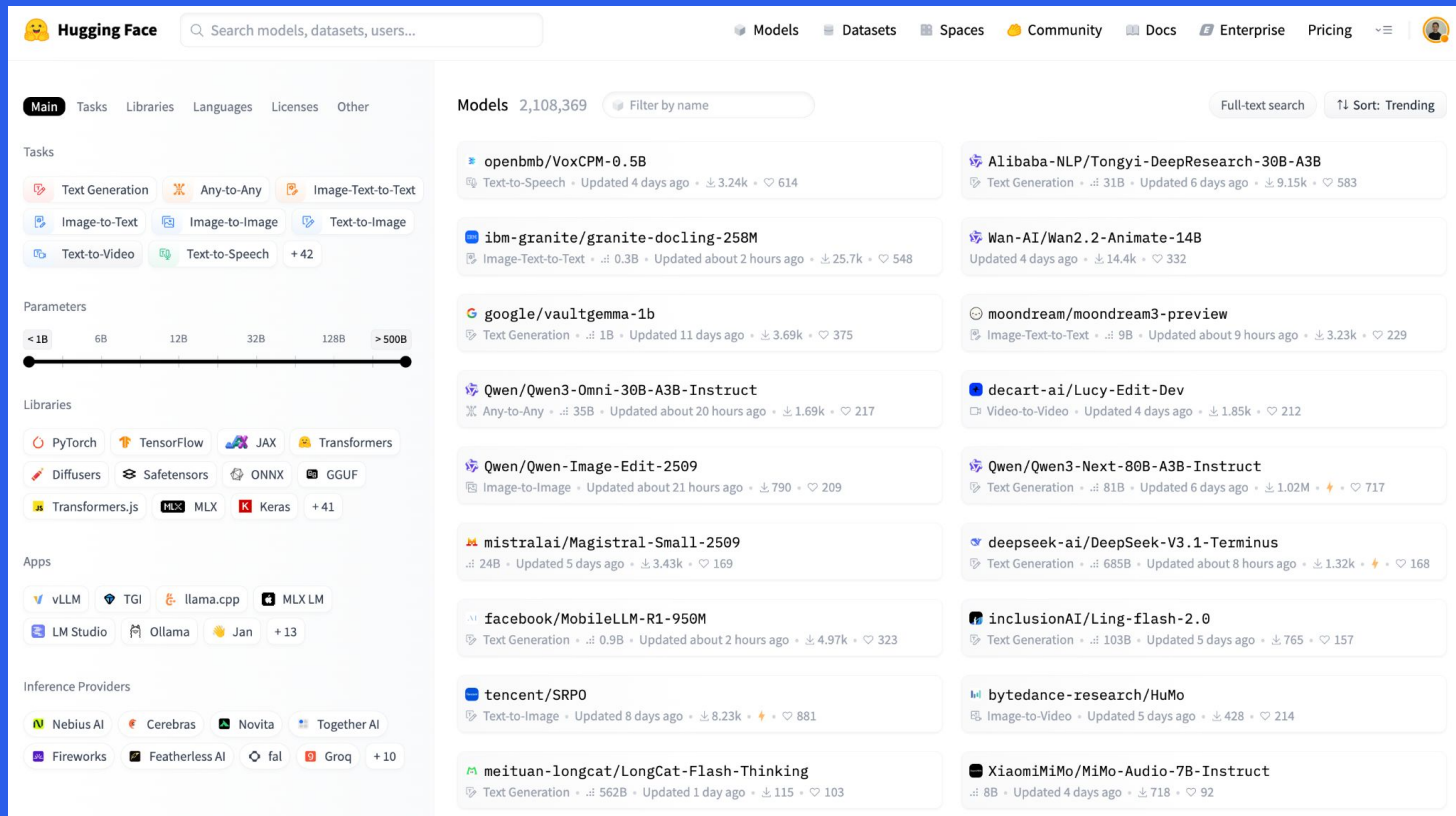


How do we build with it?

Find model → Customize → Deploy



Find the model



The screenshot displays the Hugging Face website's 'Models' section. The top navigation bar includes the Hugging Face logo, a search bar, and links for Models, Datasets, Spaces, Community, Docs, Enterprise, Pricing, and a user profile. The left sidebar features a 'Main' tab and navigation links for Tasks, Libraries, Languages, Licenses, and Other. Under 'Tasks', there are buttons for Text Generation, Any-to-Any, Image-Text-to-Text, Image-to-Text, Image-to-Image, Text-to-Image, Text-to-Video, Text-to-Speech, and a '+42' link. The 'Parameters' section shows a slider from <1B to >500B. The 'Libraries' section lists various frameworks like PyTorch, TensorFlow, JAX, Transformers, Diffusers, Safetensors, ONNX, GGUF, Transformers.js, MLX, and Keras. The 'Apps' section lists vLLM, TGI, llama.cpp, MLX LM, LM Studio, Ollama, Jan, and a '+13' link. The 'Inference Providers' section lists Nebius AI, Cerebras, Novita, Together AI, Fireworks, Featherless AI, fal, Groq, and a '+10' link. The main content area shows a list of models with filters and sorting options. The models listed include: openbmb/VoxCPM-0.5B, Alibaba-NLP/Tongyi-DeepResearch-30B-A3B, ibm-granite/granite-docling-258M, Wan-AI/Wan2.2-Animate-14B, google/vaultgemma-1b, moondream/moondream3-preview, Qwen/Qwen3-Omni-30B-A3B-Instruct, decart-ai/Lucy-Edit-Dev, Qwen/Qwen-Image-Edit-2509, Qwen/Qwen3-Next-80B-A3B-Instruct, mistralai/Magistral-Small-2509, deepseek-ai/DeepSeek-V3.1-Terminus, facebook/MobileLLM-R1-950M, inclusionAI/Ling-flash-2.0, tencent/SRPO, bytedance-research/HuMo, meituan-longcat/LongCat-Flash-Thinking, and XiaomiMiMo/MiMo-Audio-7B-Instruct. Each model entry shows its name, task, update time, size, and popularity metrics.

Hugging Face Search models, datasets, users...

Models Datasets Spaces Community Docs Enterprise Pricing

Main Tasks Libraries Languages Licenses Other

Tasks

Text Generation Any-to-Any Image-Text-to-Text

Image-to-Text Image-to-Image Text-to-Image

Text-to-Video Text-to-Speech +42

Parameters

<1B 6B 12B 32B 128B >500B

Libraries

PyTorch TensorFlow JAX Transformers

Diffusers Safetensors ONNX GGUF

Transformers.js MLX Keras +41

Apps

vLLM TGI llama.cpp MLX LM

LM Studio Ollama Jan +13

Inference Providers

Nebius AI Cerebras Novita Together AI

Fireworks Featherless AI fal Groq +10

Models 2,108,369 Filter by name

Full-text search Sort: Trending

openbmb/VoxCPM-0.5B

Text-to-Speech • Updated 4 days ago • 3.24k • 614

Alibaba-NLP/Tongyi-DeepResearch-30B-A3B

Text Generation • 31B • Updated 6 days ago • 9.15k • 583

ibm-granite/granite-docling-258M

Image-Text-to-Text • 0.3B • Updated about 2 hours ago • 25.7k • 548

Wan-AI/Wan2.2-Animate-14B

Updated 4 days ago • 14.4k • 332

google/vaultgemma-1b

Text Generation • 1B • Updated 11 days ago • 3.69k • 375

moondream/moondream3-preview

Image-Text-to-Text • 9B • Updated about 9 hours ago • 3.23k • 229

Qwen/Qwen3-Omni-30B-A3B-Instruct

Any-to-Any • 35B • Updated about 20 hours ago • 1.69k • 217

decart-ai/Lucy-Edit-Dev

Video-to-Video • Updated 4 days ago • 1.85k • 212

Qwen/Qwen-Image-Edit-2509

Image-to-Image • Updated about 21 hours ago • 790 • 209

Qwen/Qwen3-Next-80B-A3B-Instruct

Text Generation • 81B • Updated 6 days ago • 1.02M • 717

mistralai/Magistral-Small-2509

24B • Updated 5 days ago • 3.43k • 169

deepseek-ai/DeepSeek-V3.1-Terminus

Text Generation • 685B • Updated about 8 hours ago • 1.32k • 168

facebook/MobileLLM-R1-950M

Text Generation • 0.9B • Updated about 2 hours ago • 4.97k • 323

inclusionAI/Ling-flash-2.0

Text Generation • 103B • Updated 5 days ago • 765 • 157

tencent/SRPO

Text-to-Image • Updated 8 days ago • 8.23k • 881

bytedance-research/HuMo

Image-to-Video • Updated 5 days ago • 428 • 214

meituan-longcat/LongCat-Flash-Thinking

Text Generation • 562B • Updated 1 day ago • 115 • 103

XiaomiMiMo/MiMo-Audio-7B-Instruct

8B • Updated 4 days ago • 718 • 92

<https://huggingface.co/models>

Use it with 🧡 Transformers

The Transformers Library: standardizing model definitions

Published May 15, 2025

[Update on GitHub](#)



Lysandre
lysandre

Follow



Arthur Zucker
ArthurZ

Following



Pedro Cuenca
pcuenca

Following



Julien Chaumond
julien-c

Following

TLDR: Going forward, we're aiming for Transformers to be the pivot across frameworks: if a model architecture is supported by transformers, you can expect it to be supported in the rest of the ecosystem.

Transformers was created in 2019, shortly following the release of the BERT Transformer model. Since then, we've continuously aimed to add state-of-the-art architectures, initially focused on NLP, then growing to Audio and computer vision. Today, transformers is the default library for LLMs and VLMs in the Python ecosystem.



<https://huggingface.co/blog/transformers-model-definition>



Use it with 🙌 Transformers

```
from transformers import pipeline

# Load the text generation pipeline with Qwen3
pipe = pipeline(
    "text-generation",
    model="Qwen/Qwen3-0.6B",
    dtype="auto",
    device_map="auto"
)

# Define the conversation (messages)
messages = [
    {
        "role": "user",
        "content": "Give me a short introduction to large language model."
    }
]

# Generate the response
outputs = pipe(
    messages,
    max_new_tokens=512,
)

# The pipeline returns structured output
print(outputs[0]["generated_text"][-1]["content"])
...
<think>
Okay, the user wants a short introduction to a large language model. Let me start by ...
</think>

A large language model (LLM) is a type of artificial intelligence that can understand, generate, and respond to human language. These models are trained on massive datasets to understand complex topics and produce creative, context-aware content. They are used in various applications, from writing essays to providing translations and answering questions.
...
```

<https://huggingface.co/docs/transformers/index>



Use it with 🤗 Transformers

```
from transformers import AutoProcessor, AutoModelForCausalLM
import torch

model_name = "Qwen/Qwen3-0.6B"
processor = AutoProcessor.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    dtype="auto",
).to("cuda")

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "Give me a short introduction to large language model"},
        ]
    },
]

inputs = processor.apply_chat_template(
    messages,
    add_generation_prompt=True,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
).to(model.device)

generated_ids = model.generate(**inputs, do_sample=False, max_new_tokens=256)
generated_texts = processor.batch_decode(
    generated_ids,
    skip_special_tokens=True,
)

print(generated_texts[0])
...
<think>
Okay, the user wants a short introduction to a large language model. Let me start by ...
</think>
```

A large language model (LLM) is a type of artificial intelligence that can understand, generate, and respond to human language. These models are trained on massive datasets to understand complex topics and produce creative, context-aware content. They are used in various applications, from writing essays to providing translations and answering questions.



Customize with **TRL**

TRL allows you to fine-tune VLMs using different methods:
SFT, DPO, MPO, GRPO, GSPO, RLOO, and Online DPO.
PEFT and quantization support

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

Deploy model



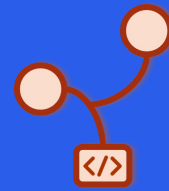
Inference and serving frameworks

Inference Providers (TGI)

Hugging Face Endpoints 

 VLLM

LLaMA 



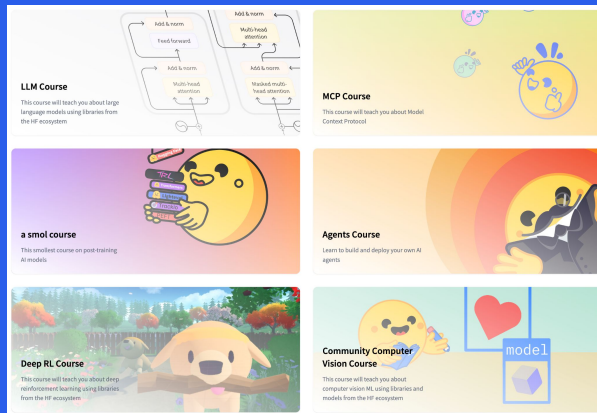
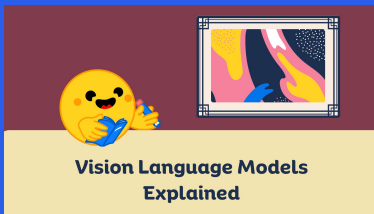
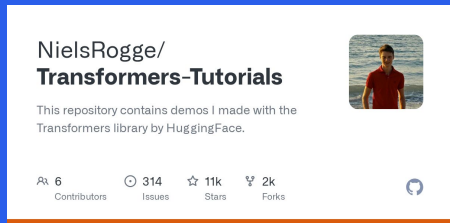
SG L



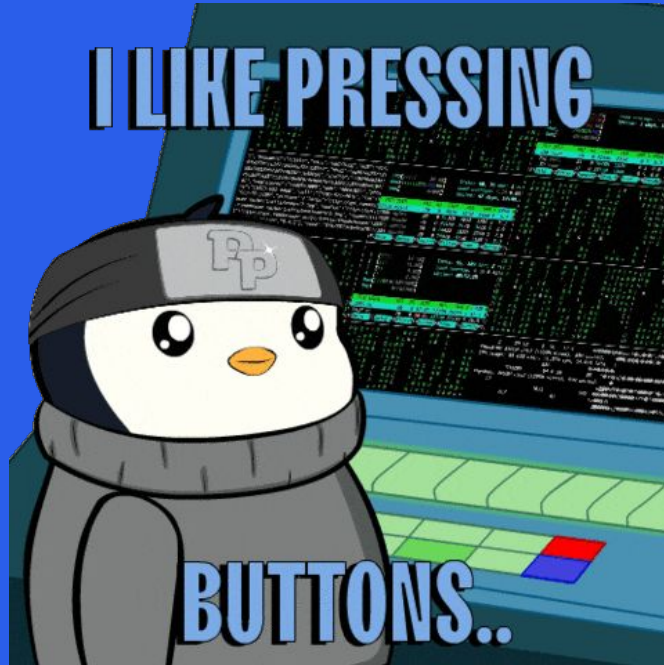
Resources



- hf.co/blog
- hf.co/learn
- [Transformers-Tutorials repo](https://github.com/NielsRogge/Transformers-Tutorials)
- [smol-vision repo](https://github.com/SmolVision/smol-vision)



Show me the code!



Notebook

Thanks!

Sergio Paniego Blanco (@sergiopaniego)

Machine Learning Engineer @ Hugging Face

